MATHRIX

User Manual

Math & Engineering Calculation Shell

Version 1.0

By Bryan Wilcutt

Contact: bwilcutt@yahoo.com

"Finally, a calculator that shows its work — and looks good doing it."

Contents

Mathrix 4
Introduction 4
What Mathrix Can Do 4
What Mathrix Can't Do 4
Installation & First Launch 5
System Requirements 5
Installation 5
First Launch 5
Interface Overview 6
The Title Bar (Header Bar) 6
The Menu Bar 6
The Splitter 7
The Status Bar 7
The Input Form Panel 7
Form Layout 8
Parameter Types 8
The Run Button 9
The Plot Button 9
Output Variable Reference 9
The Command Bar 9
Typing Commands 10
Querying Output Variables 10
Synchronizing the Form with Commands 10
The Output Panel 10
Reading the Output 10
Clearing the Output 11
Copying Results 11
Module Reference 11
Fluid Mechanics Modules 11
Hydraulics Modules 14
Structural Modules 15
Thermodynamics Modules 18
Signal Processing Modules 21
Control Systems Modules 22
Graph Window Layout 23
Graph Titles and Labels 23
Updating the Graph 24
Tips, Tricks & Troubleshooting 24
Useful Tips 24

Common Troubleshooting 24
Parse 25
What Is Parse and Why Should You Care? 25
The Prompt 28
Variables: Storing and Reusing Numbers 29
Assigning Variables 30
Built-In Math Functions 32
Engineering Commands — The Big Twelve 33
Hydraulics 34
Structural Mechanics 34
Mohr's Circle 35
Heat Conduction (Fourier's Law) 36
Specific Heat 36
Matrices: Rows, Columns, and Not Panicking 38
Matrix Arithmetic 39
Loading Data from CSV Files 40
FFT and Inverse FFT 40
Quick Reference Card 43

About this App

Math Matrix (Mathrix) was created using:

Intel i9 Computer
Linux Ubuntu/i9 Processor
Tools: MS Code, GNU tool chain, Uncrustify, ChatGPT for doc clean up.
Source code available online: https://github.com/bwilcutt/FreeCode

😅 *This program originally started as a request to write a "quick and simple" calculator for a Microchip Interview (Chandler, Az) in 2005. No, I didn't get the job. Probably because I didn't finish until 20 years later.*

Mathrix

Introduction

Welcome to Mathrix — the desktop engineering calculator that actually tells you what it just calculated and why. If you have ever stared at a black engineering textbook, laboriously punched numbers into a scientific calculator, and then spent twenty minutes checking your work because you weren't sure whether you used the right formula, Mathrix was made for you.

But what's more—if you have a formula not found in Mathrix, let us know! We are more than happy to support the community, all communities, in integrating features to make this system useful for your applications.

Mathrix is a GTK-based graphical front-end for the Parse math engine. It organizes a library of common engineering and math modules into a clean, categorized interface. You fill in the input fields, press Run, and Parse does the heavy lifting behind the scenes. If you want to see what happened visually, press Plot and a graph window pops open with a diagram of your problem — drawn to scale, labeled, color-coded, and generally much better looking than anything you would have scratched on a napkin at 2 AM.

> 😅 *The author takes no responsibility if Mathrix makes engineering look deceptively easy. It is still hard. We just made the GUI nice.*

What Mathrix Can Do

- Solve 15+ engineering modules across fluid mechanics, hydraulics, structural analysis, thermodynamics, signal processing, and control systems.
- Display interactive, color-coded graph windows for every module, showing the problem geometry and key results simultaneously.
- Support both SI and US Customary unit systems, selectable per module.
- Accept raw Parse commands via a built-in command bar, for users who prefer a scripting workflow.
- Load and analyze CSV data files, compute FFTs, and run PID controller simulations.

What Mathrix Can't Do

- Replace a licensed professional engineer. Mathrix does the arithmetic; you supply the engineering judgment. Or at least pretend to.
- Make coffee, though it does run on the same computer as your coffee-ordering app, so there is some synergy.
- Solve for variables that you did not provide. It is not psychic. Yet.

Installation & First Launch

System Requirements

Mathrix is a native Linux desktop application. It requires the following:

- A Linux distribution with GTK 4.x and GLib (Ubuntu 22.04 or later recommended).
- The Parse math engine binary, located in the same directory as the Mathrix executable (or on your PATH).
- Cairo graphics library (libcairo2), which handles all the pretty graph rendering.
- At least 50 MB of free disk space, mostly for your dignity after realizing how long you did these calculations by hand.

Installation

Mathrix is distributed as a compiled binary alongside the Parse engine. Copy the following files to a convenient directory:

```
mathrix          (the main executable)
parse            (the calculation engine)
```

Make the Mathrix binary executable if it is not already:

```
chmod +x ./mathrix
```

If you downloaded the parse manual, you can run parse independent of Mathrix. You'll be using a command line based algebraic parser at that point.

First Launch

Launch Mathrix from a terminal or file manager by running:

```
./mathrix
```

On startup, Mathrix automatically starts and connects to the Parse engine. The status bar at the bottom of the window will display "Connected to parse — ready" if everything is working correctly. If you see "ERROR: could not connect to parse," verify that the parse binary exists in the same directory and is executable.

> 💡 **Note:** *The Parse engine runs as a background subprocess. It is started automatically when Mathrix launches and is shut down cleanly when Mathrix closes. You should not need to manage it manually.*

> 😄 *If the parse binary goes missing, Mathrix will open a politely worded error dialog that is far more courteous than the situation deserves. We worked hard on that error dialog.*

Interface Overview

The Mathrix main window is divided into four horizontal zones stacked from top to bottom: the Menu Bar, the Command Bar, the Center Pane area, and the Status Bar. The center pane is itself split vertically into the Input Form Panel on the left and the Output Panel on the right.

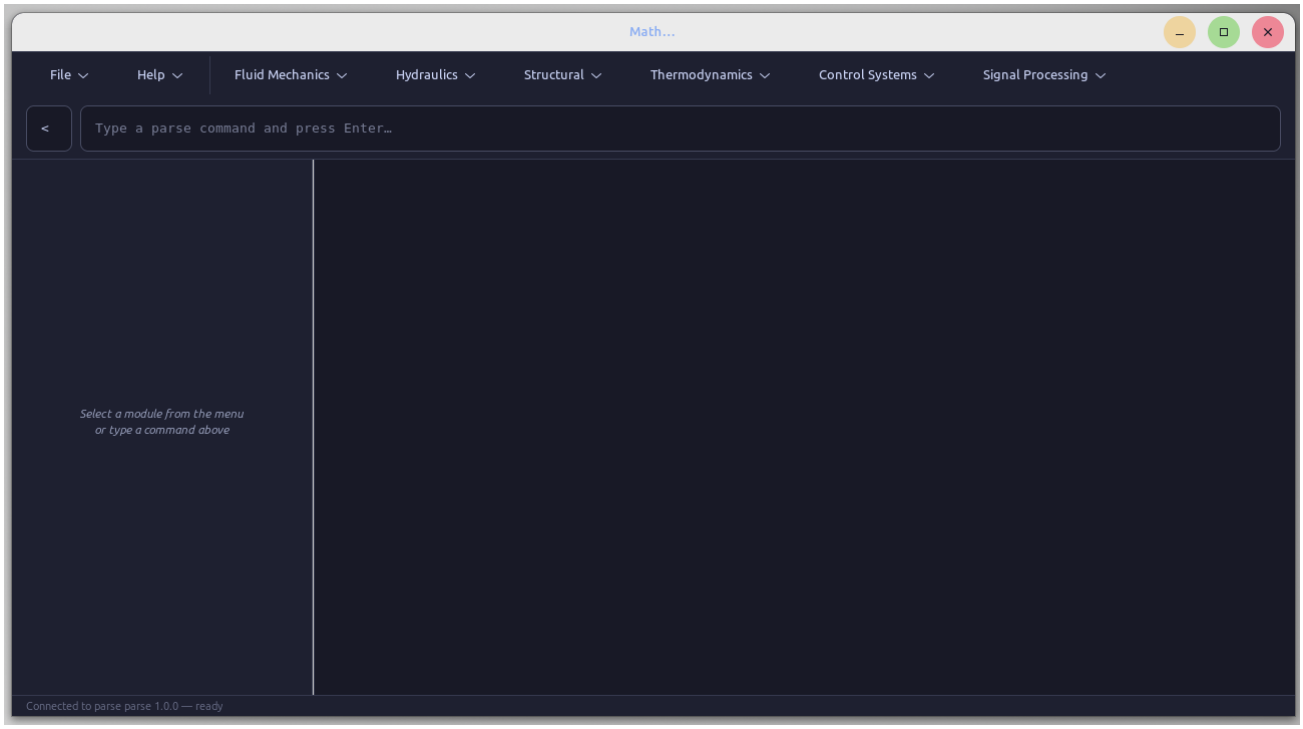


Figure 1 — The Mathrix main window with the Reynolds Number module loaded. Dark theme courtesy of Catppuccin Macchiato, which sounds like a coffee order but is actually a color scheme.

The Title Bar (Header Bar)

The header bar sits at the very top of the window and displays the application name "Mathrix" in blue. The standard window control buttons (close, minimize, maximize) appear in the top-right corner and are styled in red, yellow, and green respectively — a subtle nod to the traffic light convention and an excuse to use some of the nicer colors in the palette.

The Menu Bar

Directly below the header bar sits the menu bar. On the left are two standard menus:

- File — contains "Clear output" (which clears the output panel) and "Quit" (which does what you expect).
- Help — contains the "About Mathrix" dialog, where you can see the application version, the Parse engine version, and contact information.

To the right of File and Help, separated by a vertical divider, are module category buttons. Each category button opens a dropdown popover listing all modules in that category. The categories are:

- Fluid Mechanics — Reynolds Number, Darcy-Weisbach, Bernoulli's Equation, Orifice Flow
- Hydraulics — Manning's Equation, Orifice Flow
- Structural — Beam Deflection, Axial Stress/Strain, Mohr's Circle
- Thermodynamics — Ideal Gas Law, Heat Conduction, Specific Heat
- Signal Processing — Load CSV, FFT, Inverse FFT
- Control Systems — PID Controller

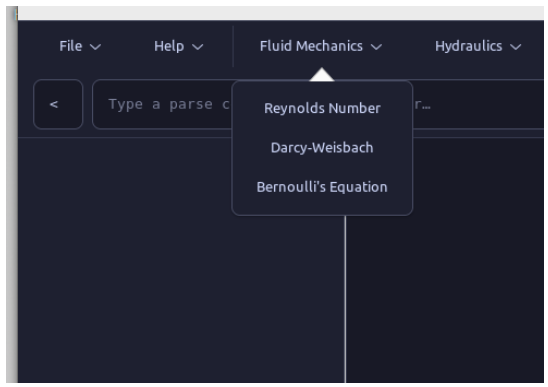


Figure 2 — The Fluid Mechanics category menu open, showing available modules.

The Splitter

The center pane can be resized by dragging the vertical splitter handle between the input form and the output panel. The default position is 300 pixels wide for the input form. If your forms feel cramped, drag it wider. If your output text is getting cut off, drag it the other direction. The splitter highlights in blue when hovered, which is the application's way of saying "yes, I am resizable, please use me."

The Status Bar

A thin bar at the very bottom of the window shows the most recent status message. It displays things like "Connected to parse — ready," "Module: Reynolds Number," or "Command sent: reynolds V=2.0 D=0.05." It is easy to ignore and easy to overlook, which is a shame because it often contains useful information about what just happened.

The Input Form Panel

The input form panel occupies the left side of the center area. When no module is selected, it displays a placeholder message inviting you to "select a module from the menu or type a command above." Once a module is selected, the form rebuilds itself entirely to show that module's inputs.

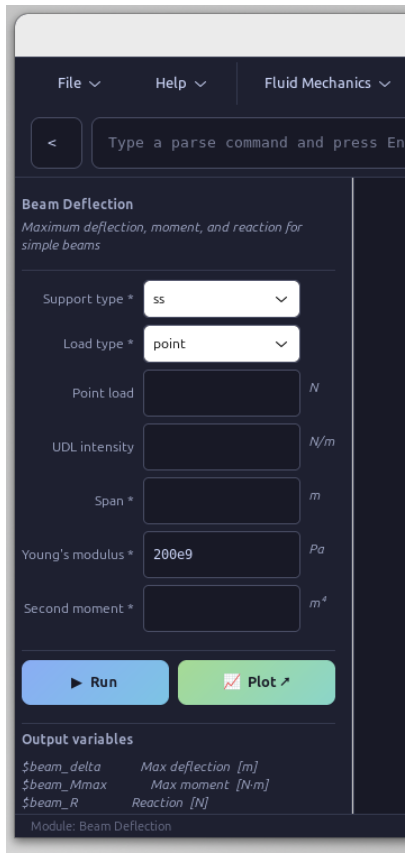


Figure 3 — The input form for Beam Deflection, showing dropdown selectors, numeric entry fields, and the Run and Plot buttons. Required fields are marked with an asterisk (*).

Form Layout

Each module's form is structured as follows, from top to bottom:

- Module title in blue, bold text at the top.
- A short description of what the module calculates, in muted italic text.
- A horizontal separator.
- A parameter grid with one row per input parameter.
- Another separator.
- The Run and Plot buttons.
- An "Output variables" section listing the variable names that Parse will populate after a successful calculation.

Parameter Types

The input form supports four types of input widgets, depending on what each parameter requires:

Numeric Entry (PT_SCALAR)

A text box accepting numeric values. Supports standard scientific notation (e.g., 200e9 for 200 gigapascals). Required fields are labeled with an asterisk. Pressing Enter in a numeric entry field triggers the Run action

automatically, which is convenient if your fingers are already on the keyboard and you don't want to reach for the mouse.

Text Entry (PT_STRING)

A text box for file paths or variable names. The file parameter type additionally shows a Browse button (📂) beside the entry field, which opens a file picker filtered to CSV files.

Dropdown (PT_CHOICE)

A dropdown selector presenting a fixed list of choices. Examples include unit systems (si / us), beam support types (ss / cant), and solve targets (which variable to compute in multi-variable equations).

Checkbox (PT_FLAG)

A checkbox for boolean flags. Currently used in the unit system selector for certain modules. When checked, the module uses US Customary units; unchecked means SI.

The Run Button

The Run button (labeled "▶ Run") is styled with a blue gradient and sends the current parameter values to the Parse engine. The results appear in the Output Panel on the right. You can also trigger Run by pressing Enter in any numeric entry field — a feature that exists because clicking buttons repeatedly is a productivity killer.

> 💡 **Note:** *All required fields (marked *) must be filled before Run will execute. If a required field is empty, the status bar will display an error message identifying the missing parameter, and the calculation will not proceed.*

The Plot Button

The Plot button (labeled "📈 Plot ↗") is styled with a green-to-teal gradient and is only visible for modules that support graphical output. Pressing Plot automatically runs the calculation first to populate all output variables, then opens a dedicated graph window showing a visual representation of the problem and results.

> 😄 *The Plot button is objectively the coolest button in the application. The Run button is fine. Very dependable. But Plot gets you graphs. Don't let Run feel bad about this.*

Output Variable Reference

Below the buttons, a monospaced list shows the output variable names that Parse will populate after a calculation. These variable names (e.g., $reynolds_Re, $beam_delta) can be used directly in the command bar to retrieve specific results, which is handy for building multi-step calculations or just for looking up a number without re-running the whole module.

The Command Bar

The command bar is the single-line text entry that sits between the menu bar and the center pane. It displays the prompt symbol "< " on the left and accepts any valid Parse command. This is the power-user interface — if you know the Parse command syntax, you can type commands directly here without touching the input form at all.

Typing Commands

Type your command in the entry field and press Enter. The command is sent to Parse, and the response appears in the output panel. For example:

```
reynolds V=2.0 D=0.05 nu=1e-6
beam type=ss load=point P=5000 L=4.0 E=200e9 I=8.33e-6
manning n=0.013 R=1.2 S=0.001 A=4.8 D=0.75
```

After typing a command, the entry field clears automatically, ready for the next command. The status bar updates to show "Command sent: [your command]."

Querying Output Variables

You can retrieve any stored Parse variable by prefixing its name with a dollar sign:

```
$reynolds_Re
$beam_delta
$mohrs_tmax
```

This is useful after running a calculation to pull out individual results, or to feed the output of one calculation as input to another. Parse stores all variables persistently across commands within the same session.

Synchronizing the Form with Commands

When you run a command via the command bar, the input form automatically re-reads the stored variable values from Parse and updates its fields to reflect them. This means the form and the command bar stay in sync — you can switch back and forth between the two interfaces freely.

> **Note:** *The command bar accepts raw Parse syntax. If you are not familiar with Parse, stick to the input form — it builds the command for you automatically and handles all the quoting and formatting that Parse expects.*

The Output Panel

The output panel occupies the right side of the center area. It is a scrollable, read-only text view displayed in monospace font on a dark background (#181926, if you are curious about the exact shade of "productive developer darkness"). This is where Parse engine responses appear after every calculation.

Reading the Output

Each calculation produces a block of text in the output panel. The format is:

```
< reynolds V=2.0 D=0.05 nu=1e-6
Re = 100000.000000
Flow regime: turbulent
```

The "< " prefix on the first line shows the command that was sent. The lines below are Parse's response. Results are printed in the order Parse computes them, which generally follows the order of the output variables listed in the module's descriptor.

Clearing the Output

The output panel accumulates results across multiple calculations within a session. To clear it, go to File > Clear output, or type Ctrl+L if you have set up that shortcut in your window manager. Starting fresh with a clean output panel is sometimes useful when switching between unrelated problems.

Copying Results

The output text is selectable. Click and drag to highlight text, then use Ctrl+C to copy it to your clipboard. This is the intended workflow for getting results out of Mathrix and into a report, spreadsheet, or email to a colleague who will inevitably ask "but did you check this with FEA?"

Module Reference

This chapter documents every calculation module available in Mathrix, organized by category. For each module, the parameter table lists the inputs, their units, whether they are required, and a brief description. The output variable names are listed separately so you can query them from the command bar.

> 😄 *There are fifteen modules here. Writing documentation for fifteen engineering formulas was, frankly, more work than writing the code for them. If you enjoy reading parameter tables, this chapter is your vacation. If not, feel free to skim and jump to whichever module you actually need right now.*

Fluid Mechanics Modules

Reynolds Number

The Reynolds number (Re = VD/v) is a dimensionless quantity that characterizes flow as laminar (Re < 2300), transitional (2300 ≤ Re ≤ 4000), or turbulent (Re > 4000). It is probably the first thing you calculate in any pipe flow problem, so naturally it gets its own module.

Parameter	Units	Required?	Description
V	m/s	Yes	Mean flow velocity
D	m	Yes	Pipe or channel diameter
Nu	m^2/s	No*	Kinematic viscosity (use instead of rho + mu)
Rho	kg/m^3	No*	Fluid density (use with mu)
mu	Pa·s	No*	Dynamic viscosity (use with rho)

* Provide either nu alone, or both rho and mu. At least one viscosity specification is required.

Output variables: $Re (Reynolds number), $reynolds_regime (0=laminar, 1=transitional, 2=turbulent)

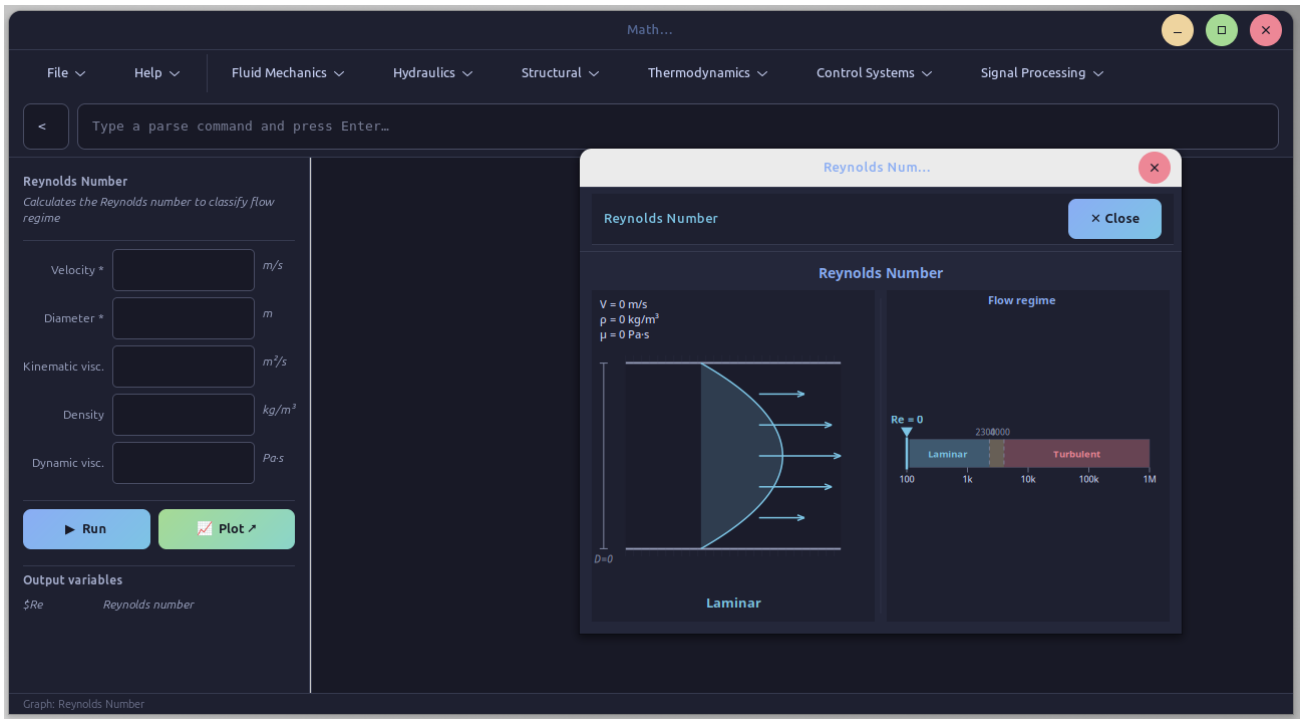


Figure 4 — Reynolds Number graph window. Left panel shows the velocity profile inside the pipe (parabolic for laminar, flattened for turbulent). Right panel shows a log-scale flow regime bar with the current Re value marked.

Darcy-Weisbach

The Darcy-Weisbach equation calculates head loss (hf) and pressure drop per unit density (ΔP/ρ) due to friction in a pipe. It requires the Darcy friction factor f, which you typically obtain from the Moody chart or a Colebrook equation solver. Mathrix does not compute f for you — that would require its own module, and the author was already tired by that point.

Parameter	Units	Required?	Description
f	—	Yes	Darcy friction factor (from Moody chart or Colebrook equation)
L	m	Yes	Pipe length
D	m	Yes	Internal pipe diameter
V	m/s	Yes	Mean flow velocity
units	—	No	Unit system: si (default) or us

Output variables: $darcy_hf (head loss, m), $darcy_dP_rho (ΔP/ρ, J/kg)

The graph window shows a pipe cross-section with a color gradient from blue (high pressure) to red (low pressure), alongside an Energy Grade Line / Hydraulic Grade Line plot showing how total head and pressure head decrease along the pipe length.

Bernoulli's Equation

Applies the Bernoulli energy equation between two cross-sections of a flow system: $P_1/\rho g + V_1^2/2g + z_1 = P_2/\rho g + V_2^2/2g + z_2$. You specify five of the six flow variables (P1, V1, z1, P2, V2, z2) and tell the module which one to solve for. It handles the algebra so you don't have to.

Parameter	Units	Required?	Description
P1	Pa	No	Upstream pressure (0 if unknown)
V1	m/s	No	Upstream velocity (0 if unknown)
z1	m	No	Upstream elevation (0 if unknown)
P2	Pa	No	Downstream pressure (0 if unknown)
V2	m/s	No	Downstream velocity (0 if unknown)
z2	m	No	Downstream elevation (0 if unknown)
Rho	kg/m³	No	Fluid density (default 1000 for water)
Solve	—	Yes	Which variable to solve for: P1, V1, z1, P2, V2, or z2

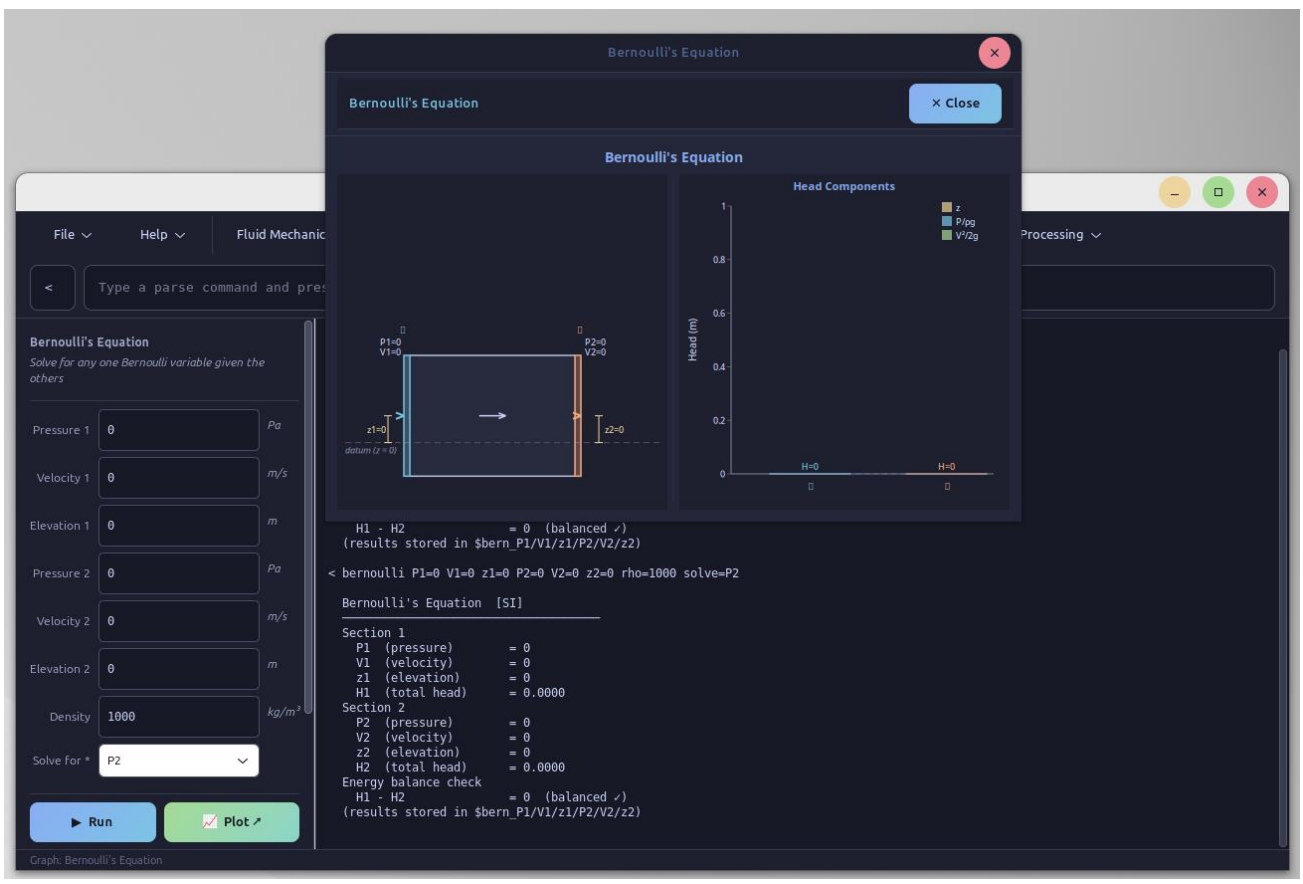


Figure 5 — Bernoulli's Equation graph window. Left panel: pipe schematic with two cross-sections at different elevations and diameters. Right panel: stacked head component bar chart comparing elevation head (z), pressure head (P/ρg), and velocity head ($V^2/2g$) at each section.

Orifice Flow

Calculates the discharge and velocity through a submerged orifice using Q = Cd·A·√(2gh), where Cd is the discharge coefficient. The default Cd of 0.61 is appropriate for a sharp-edged orifice. Use values closer to 0.98 for well-rounded, well-behaved orifices that did not grow up near sharp corners.

Parameter	Units	Required?	Description
Cd	—	Yes	Discharge coefficient (0.61 sharp-edged, 0.98 rounded)
A	m^2	Yes	Cross-sectional area of the orifice
h	m	Yes	Driving head above orifice centreline
units	—	No	Unit system: si (default) or us

Output variables: $orifice_Q (flow rate, m^3/s), $orifice_V (velocity, m/s)

The graph window shows a tank schematic with the water level, the orifice in the right wall, and the emerging jet with the vena contracta (the narrowest point of the jet) labeled. The right panel shows a Q vs. h curve with the current operating point marked.

Hydraulics Modules

Manning's Equation

Manning's equation (V = (k/n)·R^(2/3)·S^(1/2)) computes the mean velocity in an open channel. The unit conversion factor k equals 1.0 for SI and 1.486 for US customary units. Optional inputs for flow area (A) and hydraulic depth (D) allow Mathrix to also compute the discharge Q = V·A and the Froude number Fr = V/√(g·D) for regime classification.

Parameter	Units	Required?	Description
n	—	Yes	Manning's roughness coefficient (e.g., 0.013 for concrete)
R	m	Yes	Hydraulic radius = flow area / wetted perimeter
S	m/m	Yes	Longitudinal channel slope (e.g., 0.002 for 1 in 500)
A	m^2	No	Cross-sectional flow area (required for discharge Q)
D	m	No	Mean hydraulic depth A/T (required for Froude number)
units	—	No	si (default) or us (applies k = 1.486)

Output variables: $manning_V (velocity), $manning_Q (discharge), $manning_Fr (Froude number)

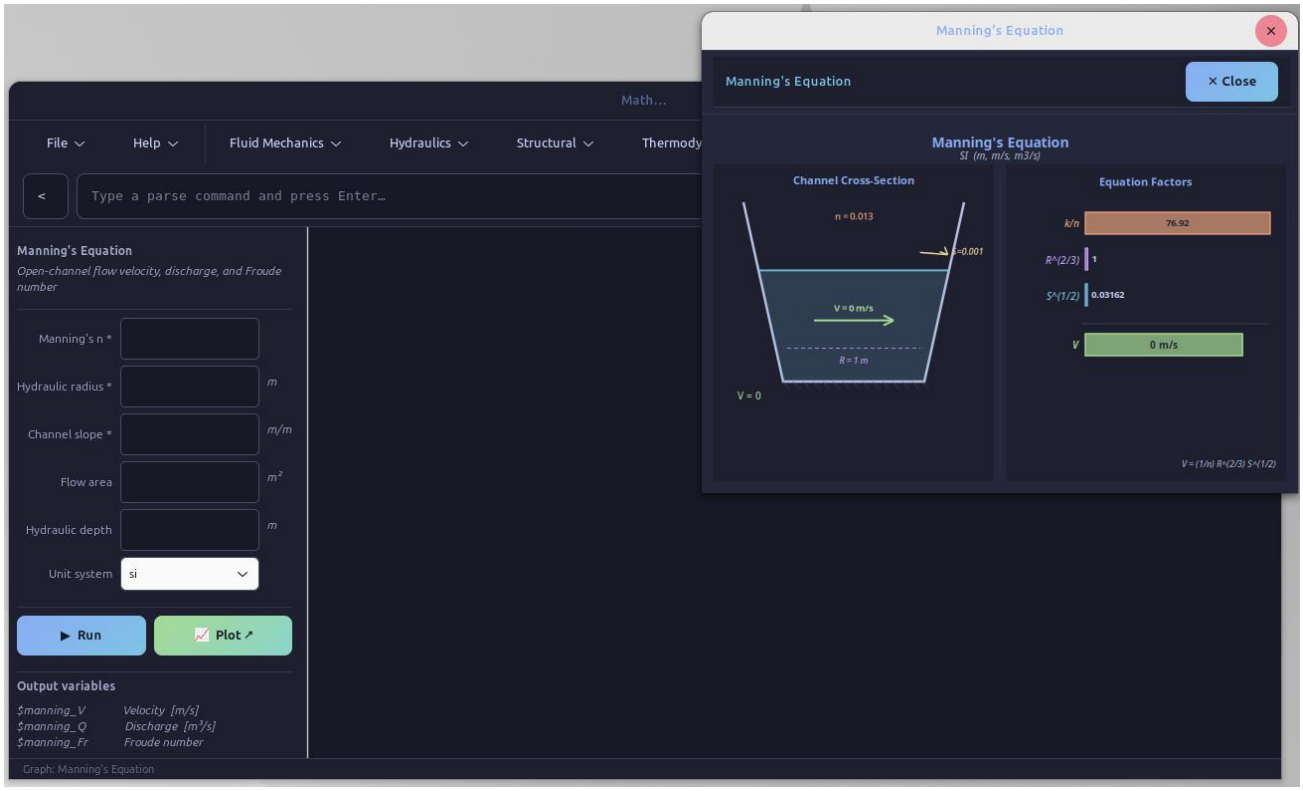


Figure 6 — Manning's Equation graph window. Left: trapezoidal channel cross-section with water surface, flow arrow, and slope indicator. Right: horizontal factor bars for k/n, R^(2/3), and S^(1/2) showing how they multiply together to produce V.

> **Note:** *The Froude number classifies flow as subcritical (Fr < 1), critical (Fr = 1), or supercritical (Fr > 1). The graph window displays the regime classification automatically when A and D are provided.*

Structural Modules

Beam Deflection

Calculates the maximum deflection, maximum bending moment, and support reactions for simple beam configurations. Supports four combinations of boundary conditions and loading: simply supported with a central point load, simply supported with a uniformly distributed load, cantilever with a tip point load, and cantilever with a full UDL.

Parameter	Units	Required?	Description
Type	—	Yes	Support type: ss (simply supported) or cant (cantilever)
Load	—	Yes	Load type: point (point load) or udl (uniform distribution)
P	N	No	Point load magnitude (required when load=point)
W	N/m	No	UDL intensity (required when load=udl)
L	m	Yes	Beam span length
E	Pa	Yes	Young's modulus (default 200 GPa for steel)
I	m^4	Yes	Second moment of area of the cross-section

Output variables: $beam_delta (max deflection, m), $beam_Mmax (max moment, N·m), $beam_R (total reaction, N)

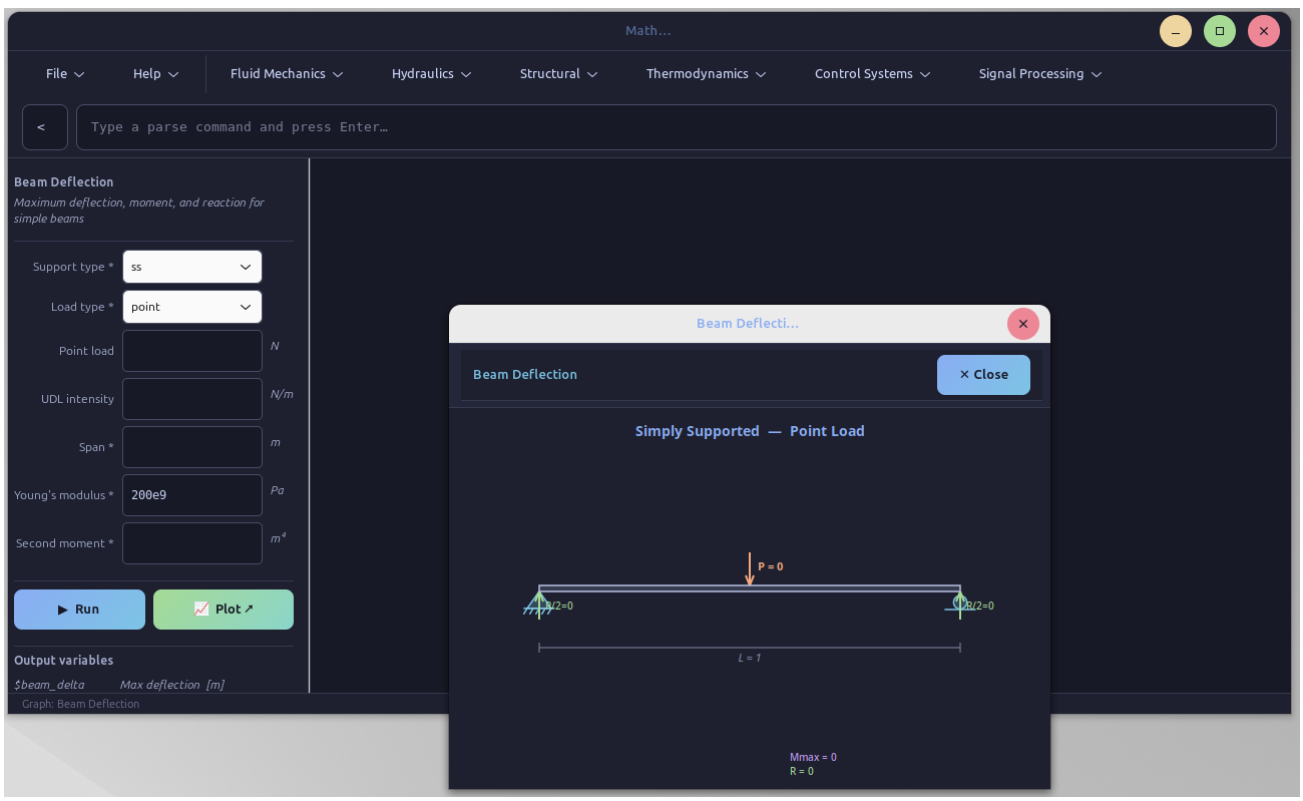


Figure 7 — Beam Deflection graph window. Shows the beam with support symbols, load arrows, reaction arrows, and an exaggerated deflected shape. The maximum deflection δ is annotated with a dashed line and arrow.

Note: *Deflections are computed using classic elastic beam formulas and shown with an exaggeration factor for visibility. The actual computed value δ is always displayed numerically regardless of scale.*

Axial Stress / Strain

Computes axial stress $\sigma = F/A$, and optionally strain $\varepsilon = \sigma/E$ and axial deformation $\delta = \varepsilon \cdot L$ for a member under axial load. Tension is positive, compression is negative. The graph window color-codes the member blue for tension and red for compression, which is a visual convention so intuitive it almost makes up for all the other conventions in structural engineering that are absolutely not intuitive at all.

Parameter	Units	Required?	Description
F	N	Yes	Axial force (positive = tension, negative = compression)
A	m^2	Yes	Cross-sectional area of the member
E	Pa	No	Young's modulus (required for strain and deformation)
L	m	No	Member length (required for axial deformation)

Output variables: $stress_sigma (Pa), $stress_eps (dimensionless), $stress_delta (m)

The graph window shows the member bar with force arrows pointing outward (tension) or inward (compression), results labeled above the bar, and — when E is provided — a σ-ε diagram on the right panel with the operating point plotted on the linear elastic line.

Mohr's Circle

Computes principal stresses (σ_1, σ_2), maximum shear stress (τmax), and principal plane orientation (θp) from a 2D stress state defined by normal stresses σx and σy and shear stress τxy. The graph window draws the actual Mohr's circle, labeled with all key points, alongside two stress element diagrams showing the original and principal orientations.

Parameter	Units	Required?	Description
sx	Pa	Yes	Normal stress on the x-face (σx)
sy	Pa	Yes	Normal stress on the y-face (σy)
txy	Pa	Yes	Shear stress on the x-face (τxy)

Output variables: $mohrs_s1 ($\sigma_1$), $mohrs_s2 ($\sigma_2$), $mohrs_tmax ($\tau max$), $mohrs_theta ($\theta p$, degrees)

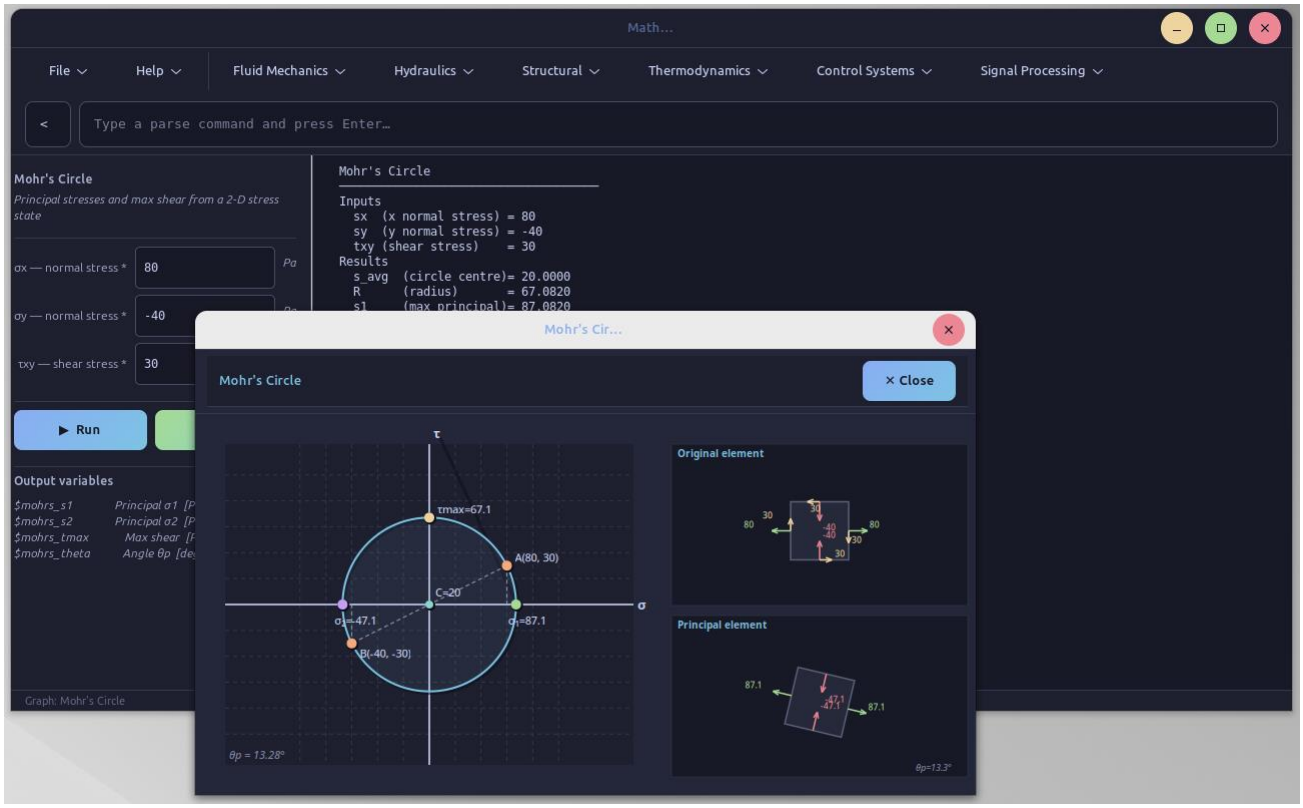


Figure 8 — Mohr's Circle graph window. Left: the circle with labeled points A (σx, τxy), B (σy, -τxy), principal stresses σ_1 and σ_2, centre C, and τmax. Right: two stress element panels showing original and principal orientations.

> 😅 *If Mohr's circle makes your head spin, that is actually the correct feeling — you are supposed to be rotating a coordinate system. The fact that it is confusing is historically accurate.*

Thermodynamics Modules

Ideal Gas Law

Solves the ideal gas law PV = nRT for any one of the four state variables P, V, n, or T, given the other three. The gas constant R is automatically set to 8.314 J/(mol·K) for SI units or 10.7316 (psi·ft³)/(lbmol·°R) for US customary units, based on the magnitude of the pressure input.

Parameter	Units	Required?	Description
P	Pa	No	Absolute pressure (leave 0 to solve for P)
V	m^3	No	Volume (leave 0 to solve for V)
n	mol	No	Amount of substance in moles (leave 0 to solve for n)
T	K	No	Absolute temperature in Kelvin (leave 0 to solve for T)
solve	—	Yes	Which variable to solve for: P, V, n, or T
units	—	No	si (default) or us

Output variables: $gas_P, $gas_V, $gas_n, $gas_T (all state variables are stored)

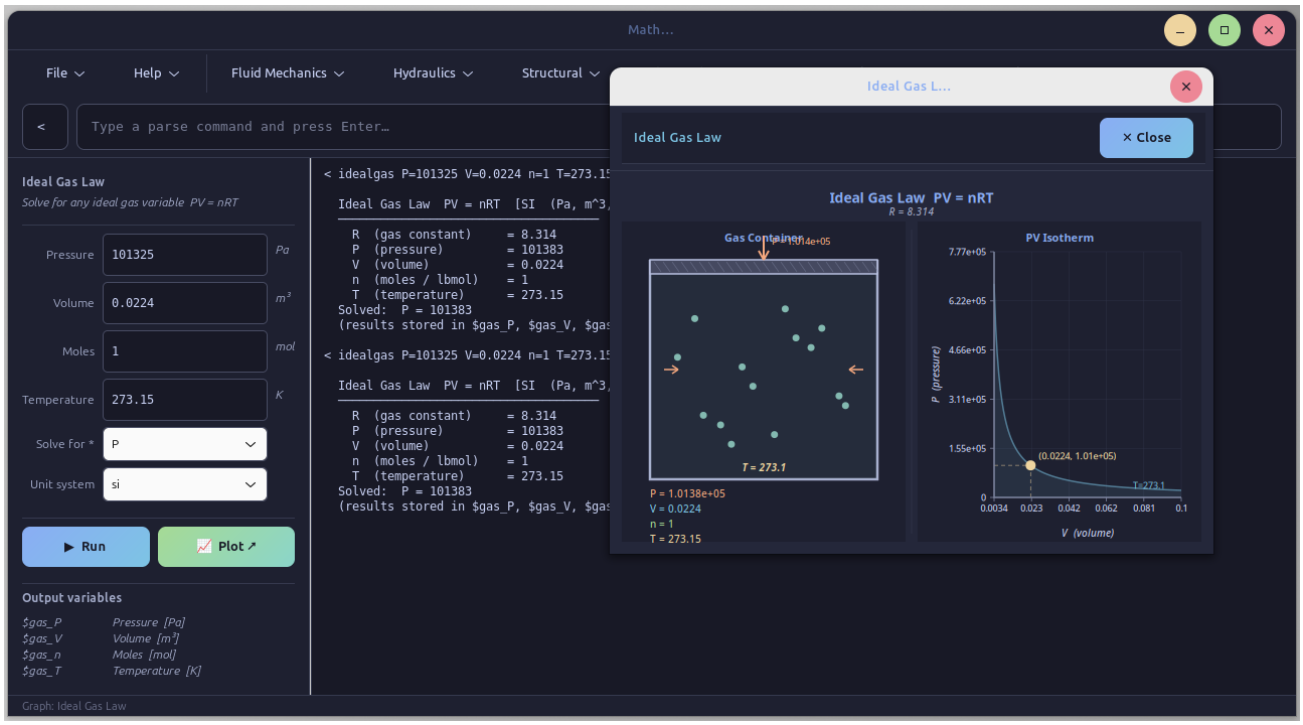


Figure 9 — Ideal Gas Law graph window. Left: container schematic with piston, pressure arrows, molecule dots, and temperature label. Right: PV isotherm curve (PV = nRT = const) with the current operating point marked as a yellow dot.

Note: *All temperatures must be in absolute units (Kelvin for SI, Rankine for US). Entering Celsius or Fahrenheit values will produce incorrect results without complaint from either Mathrix or the laws of thermodynamics.*

Heat Conduction

Applies Fourier's law of steady-state heat conduction through a planar slab: $Q = kA(T_1 - T_2)/L$. Can solve for any one of the six variables Q, k, A, L, T_1, or T_2, given the rest. The thermal conductivity k determines how quickly heat flows through the material — copper (385 W/m·K) conducts heat much more readily than wood (0.12 W/m·K), which is why we don't build kettles out of lumber.

Parameter	Units	Required?	Description
k	W/m·K	Yes	Thermal conductivity (copper 385, steel 50, glass 1.05, wood 0.12)
A	m^2	Yes	Cross-sectional area perpendicular to heat flow
L	M	Yes	Slab thickness in direction of heat flow
T1	K	Yes	Temperature of the hot face
T2	K	Yes	Temperature of the cold face
units	—	No	si (default) or us
solve	—	No	Variable to solve for: Q, k, A, L, T1, or T2 (default Q)

Output variables: $heatcond_Q, $heatcond_q (heat flux), $heatcond_Rth (thermal resistance), plus k, A, L, T1, T2

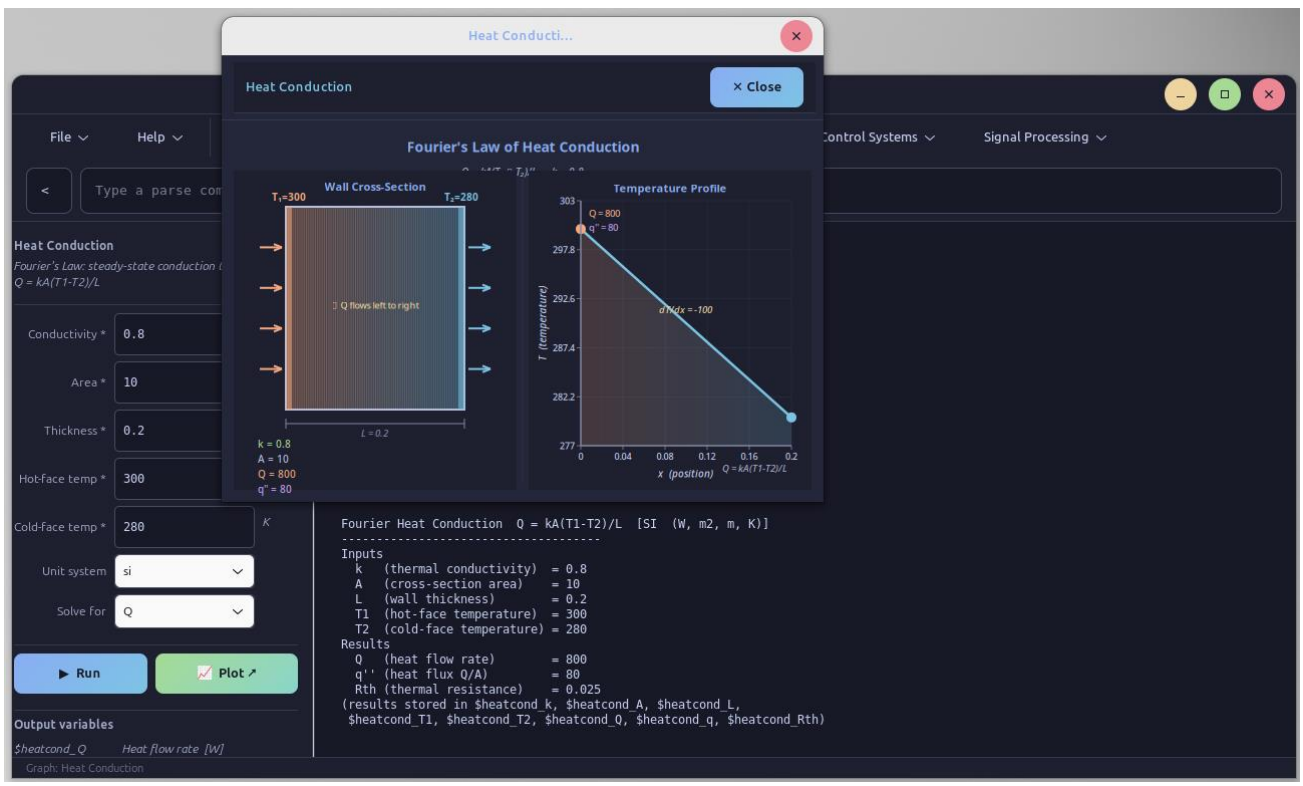


Figure 10 — Heat Conduction graph window. Left: wall cross-section with a color gradient from hot (peach/orange) on the left face to cold (sapphire/blue) on the right, with heat flux arrows. Right: linear temperature profile T(x) with the operating slope dT/dx annotated.

Specific Heat

Solves the specific heat equation Q = m·c·ΔT for any of the four variables Q, m, c, or ΔT. Useful for calorimetry problems, HVAC load calculations, or explaining to your roommate why the apartment takes so long to heat up in winter.

Parameter	Units	Required?	Description
m	Kg	No	Mass of the substance
c	J/kg·K	No	Specific heat capacity
dT	K	No	Temperature change
Q	J	No	Heat transferred (leave 0 when solving for Q)
solve	—	Yes	Which variable to solve for: Q, m, c, or dT

Output variables: $heat_Q, $heat_m, $heat_c, $heat_dT

The graph window shows a calorimeter schematic with a thermometer, and a bar chart comparing the heat energy computed for your substance against a reference bar for water at the same mass and temperature change.

> 😄 *The water reference bar exists to make you feel better or worse about your material's specific heat. Water is embarrassingly good at storing heat, which is why engineers keep using it for everything despite it being corrosive, heavy, and prone to freezing at inconvenient moments.*

Signal Processing Modules

Load CSV

Loads a column of numeric data from a CSV file into a Parse matrix variable. The loaded data can then be passed to the FFT module or plotted directly. Accepts an optional column index (default 1) and an optional output variable name (default "signal").

Parameter	Units	Required?	Description
file	—	Yes	Path to the CSV file (use Browse button to pick)
col	—	No	1-based column index to load (default 1)
var	—	No	Variable name for the loaded data (default "signal")

FFT

Computes a Cooley-Tukey radix-2 Fast Fourier Transform of a column-vector variable. The input signal length should be a power of 2 for best results. Outputs the real part, imaginary part, magnitude spectrum, and phase spectrum as separate Parse variables. The default plot shows the magnitude spectrum as a stem plot.

Parameter	Units	Required?	Description
var	—	Yes	Parse variable holding the input signal (without the $ prefix)
prefix	—	No	Prefix for output variables (default "fft")

Output variables: $fft_re, $fft_im, $fft_mag, $fft_phase

Inverse FFT

Computes the Inverse FFT from separate real and imaginary column-vector variables. Useful for reconstructing a time-domain signal after spectral processing. The output is a single recovered signal variable.

Parameter	Units	Required?	Description
re_var	—	Yes	Variable holding the real part (without $ prefix, default "fft_re")
im_var	—	Yes	Variable holding the imaginary part (default "fft_im")
prefix	—	No	Prefix for the output variable (default "ifft")

Control Systems Modules

PID Controller

The most complex module in Mathrix, the PID Controller supports four operating modes. In simulation mode (sim), it runs a closed-loop PID controller against a simple first-order plant model over a specified number of time steps, producing PV and output time series. In step mode, it computes a single PID output given the current process variable and previous state. In tune mode, it applies a selected tuning method (Ziegler-Nichols, ZN open-loop, or Cohen-Coon) to suggest gains. Reset mode clears the integrator and derivative state.

Parameter	Units	Required?	Description
mode	—	Yes	Operating mode: sim, step, tune, or reset
Kp	—	No	Proportional gain (default 1)
Ki	—	No	Integral gain (default 0)
Kd	—	No	Derivative gain (default 0)
sp	—	No	Setpoint (default 1)
dt	S	No	Time step (default 0.1)
n (sim)	—	No	Number of simulation steps (default 100)
gain (sim)	—	No	Plant gain K (default 1)
tau (sim)	S	No	Plant time constant (default 1)
method (tune)	—	No	Tuning method: zn, znol, or cc
type (tune)	—	No	Controller type: PID, PI, or P

Output variables: $pid_out, $pid_error, $pid_integral, $pid_pv, $pid_pv_vec, $pid_out_vec, and tuned gains $pid_Kp, $pid_Ki, $pid_Kd

> 😅 *The PID module has thirteen input parameters. The author is aware that this is a lot of parameters. In mitigation: PID control is genuinely complex, the alternative was a separate module for each mode, and nobody wants that.*

Graph Windows

Pressing the Plot button for any module that supports graphical output opens a dedicated graph window. Each graph window is an independent, resizable window that can remain open while you continue using the main Mathrix window. You can have multiple graph windows open simultaneously — one for your beam deflection, one for your Reynolds number, and one that you accidentally opened and are now afraid to close in case it had important information.

Graph Window Layout

Every graph window shares the same basic structure. A toolbar at the top shows the module name in sapphire-colored text. Below the toolbar is the graph canvas, which takes up the remaining window space and is rendered using Cairo graphics. When you resize the window, the diagram rescales automatically.

Most graph windows use a two-panel layout split vertically: the left panel shows a physical schematic of the problem (a beam, a pipe, a cross-section), and the right panel shows a quantitative chart or diagram (a deflection curve, a pressure-head chart, a Mohr's circle). The divider between panels is a subtle line styled in the surface color palette.

Graph Titles and Labels

Each graph window title at the top reflects the specific case being computed — for example, "Simply Supported — Point Load" for a simply supported beam with a point load, rather than just "Beam Deflection."

All axes are labeled, and key numerical results are annotated directly on the diagram to eliminate ambiguity about which number corresponds to which feature.

Updating the Graph

Graph windows don't update automatically when you change inputs in the main window. To see updated results, close the existing graph window and press Plot again after entering new values. This is intentional — it allows you to keep the previous result visible for comparison while setting up a new case.

> **Note:** *If you press Plot without first pressing Run, Mathrix runs the calculation automatically before opening the graph. You will always see the graph for the most recently computed set of inputs.*

Tips, Tricks & Troubleshooting

Useful Tips

Press Enter to Run

In any numeric input field, pressing Enter triggers the Run action. This means you can fill in fields and compute results entirely from the keyboard, without reaching for the mouse or a beer. The Plot button still requires a click, which is the application's way of asking "are you sure you want a graph?" Yes. Always yes.

Chain Calculations

Because Parse stores all computed variables persistently within a session, you can chain calculations. For example, compute a Reynolds number, then use the resulting Re to look up a friction factor elsewhere, then enter that friction factor into the Darcy-Weisbach module. The command bar is particularly useful for this workflow.

Use the Output Variable List

The "Output variables" section at the bottom of each input form lists the variable names that Parse stores after a calculation. Write these down (or just leave the form open) and type them with a $ prefix in the command bar to retrieve individual results. For example, after running the beam module, type $beam_Mmax to get just the maximum moment.

Scientific Notation in Fields

All numeric entry fields accept standard C-style scientific notation. Enter 200e9 for 200 GPa, 1e-6 for 0.000001, and so on. Don't use spaces inside the number. The form sends values directly to Parse, which handles scientific notation natively.

Keep Graph Windows for Comparison

If you are running a parametric study — say, comparing beam deflection for different span lengths — open a graph for each case and arrange them side by side on your screen. This is exactly the kind of thing that looks impressive in a presentation and takes almost no effort in Mathrix.

Common Troubleshooting

"Could not start parse binary"

This error appears if Mathrix can't find or execute the parse binary. Check that parse exists in the same directory as mathrix, is executable (chmod +x parse), and has not been accidentally renamed. Running Mathrix from a different working directory than the one containing parse can also cause this.

Graph window is blank or shows no data

This typically means a calculation produced zero or invalid output values. Check that all required fields are filled with sensible numbers. A beam with zero length or zero second moment of area will produce zero deflection, which is technically correct but also not very useful. Garbage in, garbage out — a timeless principle that predates computers by at least a few decades.

The output panel shows "(no response)"

If Parse stops responding, the output panel displays "(no response — is parse running?)." This can happen if Parse crashes due to a malformed command. Close Mathrix and relaunch it to restart the Parse subprocess. If the problem recurs consistently with a specific input, the command bar can help you identify which exact Parse command is causing trouble.

Dropdown labels showing as black boxes

On some GTK configurations, dropdown widget labels may render incorrectly. This is a known theme compatibility issue. The CSS in main_gui.c explicitly forces dropdown label colors, which resolves the problem on most systems. If you still see black rectangles where choices should be, ensure your GTK version is 4.6 or later.

> 😄 *If none of the above helps, try turning it off and on again. This advice has solved approximately 40% of all reported software issues since 1985, and there is no reason to believe Mathrix is exempt from this statistic.*

Parse

What Is Parse and Why Should You Care?

Parse is the calculation engine that lives underneath Mathrix. When you press the Run button in the Mathrix interface, it is actually Parse doing the arithmetic, looking up the formulas, and handing the answers back. Mathrix is the well-dressed front of house; Parse is the kitchen.

Normally, you never need to think about Parse at all. Mathrix handles the conversation for you — you fill in boxes, press buttons, and results appear. That is the whole point of having a graphical interface.

Because Parse can run independent of Mathrix, you can use Parse to produce your own applications and interfaces-- Afterall, Mathrix sits on top of Parse. Communication between Parse and other applications is achieved through the Pipeline Manager function. In essence, you pipe your equations to Parse and Parse returns the results using same pipe. Coherency is maintained through variable structures managed by Pipe.

😄 *Parse is a command-line program, which means it has no buttons, no menus, and no way to look up what to do when you are stuck except to type ? and read the help text. This was considered a feature in the 1970s. We have made peace with it.*

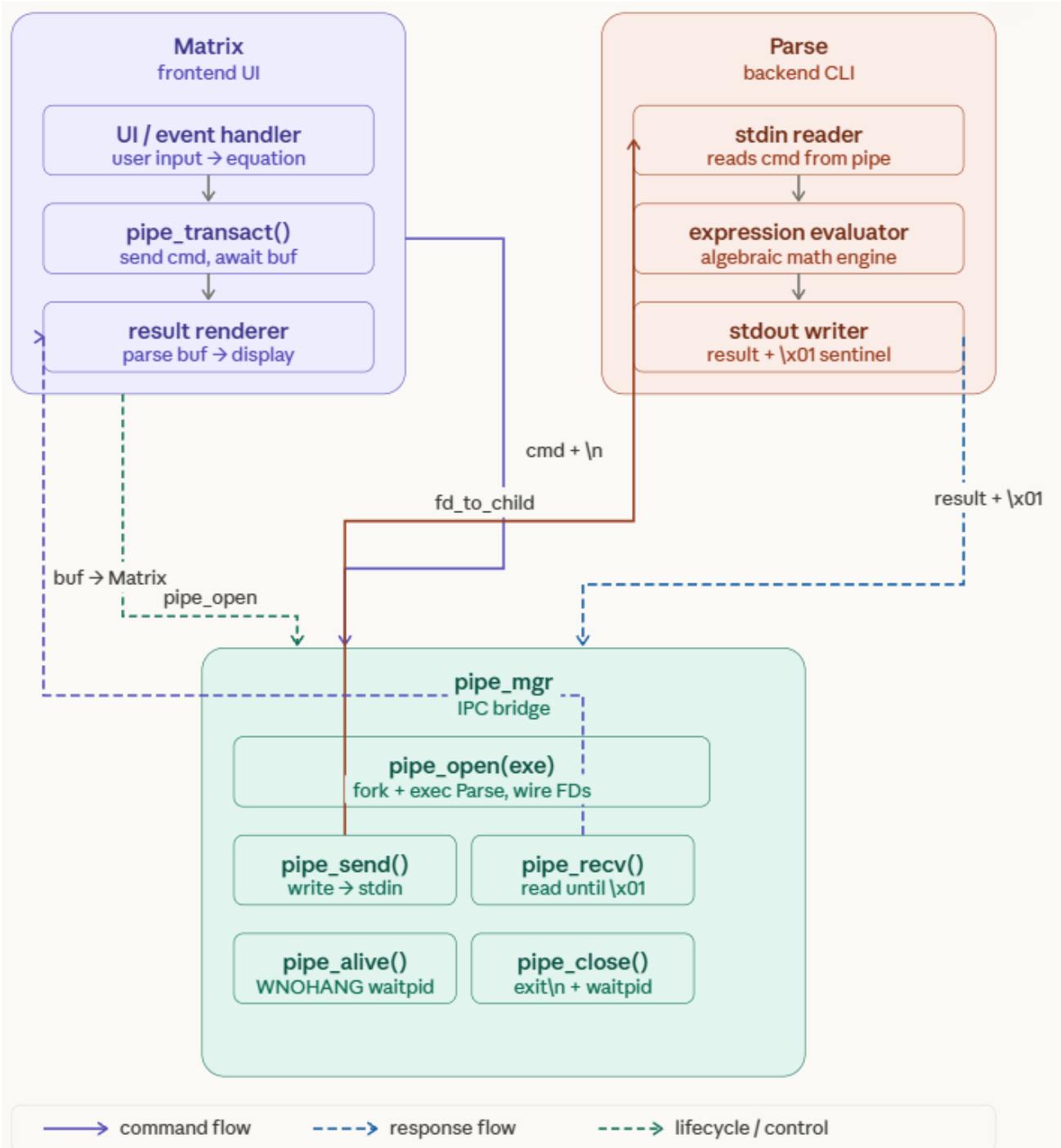
What Parse Is Not

Parse is not a spreadsheet. It does not have cells, rows, or the ability to automatically recalculate when you change a number somewhere upstream. Each command you type is evaluated once, in order, and the results are stored until you change them or close the session.

Parse is not a programming language. It does not have loops, if-statements, or user-defined functions — yet. The README is honest about this: those features are on the future expansion list. What Parse does have is more than enough for the engineering calculations Mathrix is designed to support.

Parse is not a replacement for specialist software. It will not size a structural frame, design a foundation, or simulate computational fluid dynamics. It handles the individual equations that live inside those bigger problems — the ones you used to work out by hand or reach for a calculator to do.

Here's the full picture:



The Relationship Between Mathrix and Parse

When Mathrix starts up, it launches Parse as a background process and establishes a two-way connection to it. Think of it like a very patient assistant sitting in the next room: Mathrix sends it a command, Parse works it out and replies, and Mathrix reads the reply and displays it on screen.

This arrangement means that Mathrix and Parse share a single live session. Variables you set in the command bar, or that engineering commands store automatically, persist across the entire time Mathrix is open. A Reynolds number you calculated at 9 AM is still in $Re at 4 PM, unless you have run the reynolds command again with different inputs in the meantime.

The Mathrix command bar (the text entry at the top of the window, with the "< " prompt) is a direct pass-through to Parse. Whatever you type there is sent verbatim to Parse, and Parse's response is displayed in the output panel. There is no translation, no rewriting, and no safety net — if you type something Parse does not understand, Parse will say so, and that message will appear in the output panel.

Note: *When Mathrix runs a calculation using the input form and the Run button, it constructs a Parse command automatically from your field values and sends it the same way. The command bar just lets you do that part yourself.*

The Prompt

In the command bar, you will see the "< " prompt waiting for input. This is Parse's way of saying it is ready. When Parse is busy working on a calculation, the prompt disappears and comes back when it is done. In practice, Parse is fast enough that you will rarely notice it being away.

In the output panel, commands you sent are echoed with the same "< " prefix so you can see the history of what was sent. Parse's responses appear below each command without a prefix.
Basic Arithmetic

At its simplest, Parse is a very capable calculator. You can type any arithmetic expression and it will evaluate it immediately. The four standard operators all work as expected:

Command / Syntax	What it does
2 + 2	Addition — returns 4
100 - 37.5	Subtraction — returns 62.5
9.81 * 2.5	Multiplication — returns 24.525
1000 / 4	Division — returns 250
2 * (3 + 4)	Grouping with parentheses — returns 14
-9.81	Unary negation — returns -9.81

Parse follows standard mathematical precedence: multiplication and division happen before addition and subtraction. Parentheses override this, exactly as they do in any formula. There are no surprises here — if the formula would work on a calculator, it will work in Parse.

Parse also supports scientific notation, which is essential for engineering work where numbers routinely span many orders of magnitude:

200e9 can also be 200000000000

1.5e-6 can also be 1.5e-06

You can mix scientific notation freely with regular numbers in the same expression:

5000 * 4.0^3 / (48 * 200e9 * 8.33e-6) is 0.00200048

😄 *That expression above is a beam deflection calculation, and it is exactly the kind of thing you would have typed into a scientific calculator one painful number at a time. Now you can type the whole formula and let Parse handle the button-pressing.*

```
Math Shell (type '?' for help, 'exit' to quit)
<  1 + 1
2
<  3.14159 * 2^2
12.5664
<  sqrt(144)
12
<  sin(3.14159/2)
1
<  _
```

Figure A.1 — A basic Parse session showing arithmetic, square roots, and trig. The green text is what you type; the white text below each line is Parse's response.

💡 **Note:** *Division by zero does not crash Parse — it will return infinity or produce an error message. It is polite about it however a rumor has spread around that it'll open a time-space rift. Berenstain-Berestein.*

Variables: Storing and Reusing Numbers

Variables are one of the most useful features in Parse, and one of the most important things to understand if you want to get the most out of Mathrix. A variable is simply a named container for a number. You give it a name, assign a value, and then use that name wherever you need the value — in calculations, in engineering commands, anywhere.

Assigning Variables

Variable names always start with a dollar sign ($). This is how Parse knows you are referring to a variable rather than a command or a number. To assign a variable, write its name, an equals sign, and an expression:

```
< $pipe_diameter = 0.05
  0.05

< $flow_velocity = 2.0
  2
```

The value on the right can be any valid expression — it does not have to be a plain number:

```
< $area = 3.14159 * 0.025 * 0.025
  0.00196349

< $half_span = $total_length / 2
  (whatever $total_length / 2 evaluates to)
```

```
<  $radius = 5
5
<  $area = 3.14159 * $radius * $radius
78.5398
<  $circumference = 2 * 3.14159 * $radius
31.4159
<  vars
Scalar variables:
$area                = 78.5398
$circumference       = 31.4159
$radius              = 5

<  _
```

Figure A.2 — Variable assignment and the "vars" command. Once set, variables persist for the entire Mathrix session.

Using Variables in Expressions

Once a variable is set, you can use it by name — with the $ prefix — in any expression or engineering command:

```
< $pipe_diameter * 1000
```

```
 50   (diameter in mm)

< reynolds V=$flow_velocity D=$pipe_diameter nu=1e-6
```

That last line is the key insight: Parse engineering commands accept expressions as parameter values. This means you can store your inputs in variables, run the calculation using those variables, change one variable, and re-run the calculation instantly without retyping everything. This is the closest thing Parse has to the spreadsheet's automatic recalculation, and it is genuinely useful for parametric studies.

Variables Set by Engineering Commands

Every engineering command in Parse automatically stores its results in a set of named variables. These are listed at the bottom of each command's output — the lines that read "(results stored in $varname...)". You don't need to do anything special to capture results; they are stored automatically every time you run a command.

For example, after running the reynolds command, the result is available as $Re:

```
< reynolds V=2.0 D=0.05 nu=1e-6
< $Re
 100000
```

You can then use $Re in a subsequent calculation or even pass it as a parameter to another command. The results from one calculation become the inputs for the next.

Listing All Variables

To see every variable currently stored in the session, type "vars" at the prompt. This shows a formatted table of all scalar variable names and their current values. It is useful when you have been running multiple calculations and want to check what is stored before using a value in an expression.

```
vars
 Scalar variables:
  $Re                = 100000
  $reynolds_regime      = 2
  $pipe_diameter        = 0.05
  $flow_velocity        = 2
```

Note: *Variable names are case-sensitive. $Re and $re are different variables. The engineering command outputs always use the exact names shown in the documentation (e.g. $Re, $beam_delta, $darcy_hf), so use those exact names when retrieving results.*

Built-In Math Functions

Parse includes a set of standard mathematical functions that can be used anywhere a number or expression is valid. The syntax is the function name followed by its argument in parentheses — no spaces needed, though they are accepted.

Command / Syntax	What it does
sin(x)	Sine — argument in radians
cos(x)	Cosine — argument in radians
tan(x)	Tangent — argument in radians
sqrt(x)	Square root — equivalent to x^0.5
abs(x)	Absolute value — strips the sign from x
log(x)	Natural logarithm (base e) — not base 10
exp(x)	e raised to the power x — inverse of log
floor(x)	Round down to nearest integer
ceil(x)	Round up to nearest integer
Log10(x)	Returns the log x^{10} of the number x.

A few practical notes worth highlighting:

- Trig functions use radians. To convert degrees to radians, multiply by π/180. Since Parse does not have a built-in π constant, use 3.14159265 — or better yet, $pi = 3.14159265 at the start of your session and use $pi thereafter.
- log() is the natural logarithm. For base-10 log, compute log(x) / log(10).
- sqrt() is just a convenience. sqrt($area) and $area * 0.5^0.5... actually, just use sqrt(). It is clearer.

Functions can be nested and combined with arithmetic freely:

```
< sqrt(3^2 + 4^2)
  5   (Pythagorean theorem)

< $pi = 3.14159265
< sin($pi / 6)
  0.5   (sin 30°)
```

< log(exp(1))

1 (as it should be)

Engineering Commands — The Big Twelve

The engineering commands are what set Parse apart from a plain calculator. Each command encodes a specific formula or set of formulas, validates your inputs, does the math, prints a formatted summary, and stores all the results in named variables for future use. They are listed here grouped by discipline, with the essential syntax and result variables for each.

All engineering commands accept expressions — not just plain numbers — as parameter values. Anywhere you see <val> in the syntax, you can also write a variable name, an arithmetic expression, or a combination of both.

Fluid Mechanics

Reynolds Number

Classifies pipe or channel flow as laminar, transitional, or turbulent using Re = VD/v.

< reynolds V=<val> D=<val> nu=<val>

OR use density and dynamic viscosity instead of kinematic viscosity:

< reynolds V=<val> D=<val> rho=<val> mu=<val>

Result variables: $Re (Reynolds number), $reynolds_regime (0=laminar, 1=transitional, 2=turbulent)

```
<  reynolds V=2.0 D=0.05 nu=1e-6

Reynolds Number
──────────────────────────────────
Inputs
V   (velocity)              = 2
D   (characteristic length) = 0.05
nu  (kinematic visc)        = 1e-06
Result
Re                          = 100000.00  (turbulent)

(result stored in $Re, $reynolds_regime)

<  $Re

100000

<  $pipe_vel = 3.5

3.5

<  reynolds V=$pipe_vel D=0.05 nu=1e-6

...
<
```

Figure A.3 — Reynolds Number session. After the first calculation, $Re holds the result. The third command shows how to use a stored variable as a parameter value in a new run.

Darcy-Weisbach

Computes friction head loss in a pipe: $hf = f(L/D)V^2/2g$. Requires the Darcy friction factor f, which you look up from a Moody chart based on Re and relative roughness.

< darcy f=<val> L=<val> D=<val> V=<val> [units=si|us]

Result variables: $darcy_hf (head loss), $darcy_dP_rho (ΔP/ρ in J/kg)

Bernoulli's Equation

Solves the Bernoulli energy equation between two pipe cross-sections, computing whichever of the six variables (P1, V1, z1, P2, V2, z2) you specify as the unknown. Supply the other five and tell it which to solve for.

< bernoulli P1=<> V1=<> z1=<> P2=<> V2=<> z2=<> [rho=<>] solve=<P1|V1|z1|P2|V2|z2>

Result variables: $bern_P1, $bern_V1, $bern_z1, $bern_P2, $bern_V2, $bern_z2

Orifice Flow

Computes discharge and velocity through an orifice: $Q = Cd·A·√(2gh)$. Cd = 0.61 is the default for a sharp-edged orifice.

< orifice Cd=<val> A=<val> h=<val> [units=si|us]

Result variables: $orifice_Q (discharge), $orifice_V (jet velocity)

Hydraulics

Manning's Equation

Open-channel flow velocity using $V = (k/n)·R^{(2/3)}·S^{(1/2)}$. k = 1.0 for SI, 1.486 for US. Optionally computes discharge Q and Froude number Fr when area and hydraulic depth are supplied. The special command "manning ?" prints a reference table of roughness coefficients for common channel types — a genuinely useful lookup that saves you hunting through a handbook.

< manning n=<val> R=<val> S=<val> [A=<val>] [D=<val>] [units=si|us]

manning ?

Prints a roughness coefficient reference table

Result variables: $manning_V, $manning_Q (if A given), $manning_Fr (if A and D given)

Structural Mechanics

Beam Deflection

Classic closed-form beam formulas for four configurations: simply supported or cantilever, with a point load or uniform distributed load. Gives maximum deflection, maximum moment, and total reaction. Also checks the L/360 serviceability limit and warns you if you have exceeded it.

< beam type=ss|cant load=point|udl P=<>|w=<> L=<> E=<> I=<>

Result variables: $beam_delta (max deflection), $beam_Mmax (max moment), $beam_R (total reaction)

```
# Set up beam parameters as variables first
<  $span = 4.0
<  $load = 5000
<  $Emod = 200e9
<  $Ixx = 8.33e-6
<  beam type=ss load=point P=$load L=$span E=$Emod I=$Ixx
Beam Deflection  [Simply Supported — Point Load]
──────────────────────────────────────
P  (point load)        = 5000
L  (span)              = 4
delta (max deflection) = 0.00200024  (at midspan)
M  (max moment)        = 5000       (at midspan)
R  (total reaction)    = 5000
(results stored in $beam_delta, $beam_Mmax, $beam_R)
<  $beam_delta * 1000
2.00024   (deflection in mm)
<
```

Figure A.4 — Beam calculation using stored variables for parameters. The last line converts the deflection from metres to millimetres by multiplying the stored result by 1000. This is the kind of chaining that makes Parse genuinely useful.

Axial Stress / Strain

Computes axial stress σ = F/A. With Young's modulus E, also computes strain ε = σ/E. With member length L as well, also computes axial deformation δ = ε·L. Positive force is tension; negative is compression.

< stress F=<val> A=<val> [E=<val>] [L=<val>]

Result variables: $stress_sigma, $stress_eps (if E given), $stress_delta (if E and L given)

Mohr's Circle

Computes principal stresses, maximum shear stress, and principal plane angle from a 2D stress state. All three inputs are required; units are whatever you supply (Pa, MPa, ksi — Parse does not care as long as they are consistent).

< mohrs sx=<val> sy=<val> txy=<val>

Result variables: $mohrs_s1, $mohrs_s2, $mohrs_tmax, $mohrs_theta (degrees)

Thermodynamics

Ideal Gas Law

Solves PV = nRT for any one of the four state variables. Tell it which to solve for with the solve= parameter. R is set automatically based on the unit system (8.314 J/mol·K for SI, 10.7316 psi·ft³/lbmol·°R for US).

< idealgas P=<> V=<> n=<> T=<> [units=si|us] solve=<P|V|n|T>

Result variables: $gas_P, $gas_V, $gas_n, $gas_T (all four are always stored)

Heat Conduction (Fourier's Law)

Solves $Q = kA(T_1-T_2)/L$ for any one variable. The default behaviour — when you provide all five of k, A, L, T1, T2 without specifying solve= — is to compute Q. To solve for a different variable, add solve=k, solve=L, solve=T1, etc. Also computes heat flux q''= Q/A and thermal resistance R_th = L/(kA).

< heatcond k=<> A=<> L=<> T1=<> T2=<> [units=si|us] [solve=<k|A|L|T1|T2>]

Result variables: $heatcond_Q, $heatcond_q, $heatcond_Rth, plus $heatcond_k, $heatcond_A, $heatcond_L, $heatcond_T1, $heatcond_T2

Specific Heat

Solves Q = m·c·ΔT for any of the four variables. When m, c, and dT are all given without a solve= parameter, it computes Q automatically. Common specific heat values: water = 4186 J/kg·K, steel = 490, aluminium = 900, copper = 385.

```
< specheat m=<val> c=<val> dT=<val>
  # Solves Q automatically when all three are given
< specheat solve=<Q|m|c|dT> ...
  # Solve for a specific variable
```

Result variables: $heat_Q, $heat_m, $heat_c, $heat_dT

😄 *The specheat command is the only one that defaults to solving Q without requiring solve=. This inconsistency was noticed during testing and left in anyway, because changing it would break the Mathrix GUI, which was already written. Welcome to the real world of software.*

Chaining Calculations Together

One of the most powerful things you can do with Parse is chain calculations — using the output of one command as the input to another. Because every engineering command stores its results in named variables, and because every parameter value in a command can be an expression, these two features combine naturally.

A Worked Example: Pipe Flow Analysis

Suppose you want to analyze a pipe, starting with the flow regime and then computing the head loss. A chained workflow might look like this:

```
 # Step 1: set up the pipe parameters
< $D = 0.08
< $V = 1.5
< $L = 150
< $nu = 1.004e-6

 # Step 2: check the flow regime
reynolds V=$V D=$D nu=$nu
   Re  = 119522.00  (turbulent)

 # Step 3: look up friction factor from Moody chart
 #       (Parse can't do this for you — this is where the handbook lives)
< $f = 0.018

 # Step 4: compute head loss using the stored velocity and diameter
< darcy f=$f L=$L D=$D V=$V
   hf  = 36.2156 m

 # Step 5: retrieve just the head loss for use in a subsequent calc
< $darcy_hf
 36.2156
```

Notice that in Step 4, the parameters L, D, and V are passed as variable names rather than numbers. This means if you change $V in Step 2 and re-run, Step 4 automatically uses the new velocity value — you don't have to retype anything.

Using Results in Arithmetic

Stored result variables can be used in arbitrary arithmetic expressions, not just passed to other commands:

```
< $beam_delta * 1000
 2.00048   (deflection in mm)
```

```
< $Re / 2300
  43.7   (how many times above the laminar limit)

< $darcy_hf * 9810
  355274   (head loss in Pascals, for water at 9810 N/m³)
```

Note: *Parse does not know the units of your variables — that is your job. If you multiply a head loss in metres by a specific weight in N/m³, you get pressure in Pascals. If you multiply it by the wrong number, you get a confident-looking wrong answer. Parse will not warn you about this. It is doing exactly what you told it to do.*

Matrices: Rows, Columns, and Not Panicking

Parse supports matrices — rectangular arrays of numbers. If you went to engineering school, you have used matrices. If you did not, a matrix is just a table of numbers arranged in rows and columns. Parse lets you create them, add and subtract them, multiply them, transpose them, and invert them.

For most day-to-day Mathrix use, you will not need matrices at all — the engineering commands work with plain numbers. Matrices become relevant if you are doing structural analysis with multiple load cases, solving systems of simultaneous equations, or working with signal processing data loaded from a CSV file.

Creating a Matrix

Matrix literals are enclosed in square brackets. Separate values within a row with spaces or commas. Separate rows with semicolons:

```
< $A = [1 2 3; 4 5 6; 7 8 9]
  Matrix 3x3:
    1  2  3
    4  5  6
    7  8  9

< $v = [10; 20; 30]
  Matrix 3x1:
    10
    20
    30
```

The second example is a column vector — a matrix with one column. Parse does not have a separate "vector" type; vectors are just matrices with one row or one column.

Matrix Arithmetic

The +, -, and * operators work on matrices. Addition and subtraction are element-wise and require matching dimensions. Multiplication is full matrix multiplication — not element-wise — so dimensions must be compatible (the number of columns in the first matrix must equal the number of rows in the second).

```
<  $A = [1 2; 3 4]

Matrix 2x2:
1  2
3  4

<  $B = [5 6; 7 8]

Matrix 2x2:
5  6
7  8

<  $A * $B

Matrix 2x2:
19  22
43  50

<  det($A)

-2
```

Figure A.5 — Matrix creation, multiplication, and determinant. The 2×2 matrix multiply and det() calls shown here are the most common matrix operations needed in engineering work.

Matrix Functions

Command / Syntax	What it does
transpose($A)	Flip rows and columns. A 3×2 matrix becomes 2×3.
inv($A)	Matrix inverse. Square matrices only. Fails if singular.
scale($A, k)	Multiply every element by scalar k.
det($A)	Compute the determinant. Prints as a scalar value.

A practical application: solving a system of linear equations Ax = b. If you have a 3×3 stiffness matrix $K and a load vector $F, the displacement vector is inv($K) * $F:

```
< $K = [4 -1 0; -1 4 -1; 0 -1 4]
< $F = [100; 0; 100]
< $x = inv($K) * $F
  Matrix 3x1:
    37.5
    25
```

Listing Matrix Variables

Type "mats" to see all currently stored matrix variables, their dimensions, and their contents. This is the matrix equivalent of "vars" for scalar variables.

Note: *Matrix variable names follow the same $ convention as scalar variables. The set_mat_var system* used internally by Parse is separate from the scalar variable system — a $signal matrix and a $signal scalar can technically coexist, but naming things that way will confuse everyone including you. *What were we talking about again… ?*

Loading Data from CSV Files

Parse can load columns of numeric data from CSV files into matrix variables. This is the starting point for any signal processing workflow — load your data, inspect it, run an FFT, examine the results.

The load Command

The load command reads a CSV file and stores one column as a matrix variable. It auto-detects the delimiter (comma, tab, or space), and skips a header row if the first row contains text rather than numbers.

```
< load mydata.csv
  # Loads column 1 into $signal
< load mydata.csv col=2 var=pressure
  # Loads column 2 into $pressure
```

After loading, the variable contains a column vector (an N×1 matrix) with one entry per row of data. You can inspect it with "mats" and pass it to other commands.

FFT and Inverse FFT

Once data is loaded, you can compute its frequency spectrum using the fft command. Parse implements the Cooley-Tukey radix-2 FFT algorithm — the standard approach that you will find in every signal processing textbook. If the input length is not a power of 2, Parse zero-pads it automatically.

```
load samples.csv
  # Loads data into $signal
fft $signal
  FFT: 256 samples → $fft_re  $fft_im  $fft_mag  $fft_phase
```

The fft command produces four output vectors: real part, imaginary part, magnitude spectrum, and phase spectrum. For most engineering purposes, the magnitude spectrum ($fft_mag) is what you want — it tells you how much energy is present at each frequency bin.

The Mathrix GUI plots $fft_mag automatically as a stem plot when you use the FFT module from the input form. Using the command bar directly gives you access to all four output vectors separately.

To reconstruct a time-domain signal from its frequency components:

```
ifft $fft_re $fft_im
  IFFT: 256 points → $ifft_out
```

😄 *The FFT is one of the most important algorithms ever invented, and Parse quietly implements it in about 80 lines of C. If you run it and look at the output and it does not immediately make sense, don't worry — that is what the Mathrix magnitude spectrum plot is for.*

Useful Shell Commands

In addition to the engineering commands and arithmetic, Parse has a small set of housekeeping commands that are useful to know.

Command / Syntax	What it does
?	Prints the full command reference — every command, every parameter, all result variables. Long but comprehensive.
Vars	Lists all currently stored scalar variables and their values.
Mats	Lists all currently stored matrix variables, their dimensions, and contents.
Version	Prints the Parse version number. Mathrix also shows this in the About dialog.
manning ?	Special case: prints the Manning's n roughness coefficient reference table. Very handy.
Exit	Closes the Parse session. From Mathrix, just close the window instead — Mathrix sends exit automatically on shutdown.

Comments

Lines beginning with # are treated as comments and ignored. This is useful if you are building up a sequence of commands in a text file and want to annotate what each section does. You can pipe a text file of commands to Parse directly from the terminal:

```
parse < my_calculations.txt
```

This workflow — writing your calculation sequence in a text file and running it in batch — is a powerful way to document and reproduce engineering calculations. The README for Parse mentions this explicitly, and it is worth knowing about even if Mathrix handles most of your day-to-day interaction.

```
<  ?

Math Shell — command reference
──────────────────────────────────────────────────

Scalar operators
+  -  *  /          arithmetic
( )                 grouping

Scalar functions
sin(x)  cos(x)  tan(x)    trig
sqrt(x) abs(x)            root / absolute value
log(x)  exp(x)            natural log / exponential

Scalar variables
$name = expr        assign
$name               use in an expression

... (continues)

<  _
```

Figure A.6 — The help output from typing "?" in Parse. The full reference is longer than this — it scrolls. Think of it as a safety net that is always one keystroke away.

Parse Expressions as Parameter Values

One feature worth repeating because it is easy to overlook: every parameter value in every engineering command can be any valid Parse expression, not just a plain number. This means you can write things like:

```
manning n=0.013 R=$width*$depth/($width+2*$depth) S=1/500
beam L=6 E=200e9 I=$b*$h^3/12 P=$total_load/2
heatcond k=385 A=3.14159*0.025*0.025 L=0.1 T1=500 T2=20
```

In the first example, the hydraulic radius is computed inline from width and depth variables rather than being calculated separately first. In the second, the second moment of area for a rectangular section is computed inline. This is not just convenient — it makes the command self-documenting, showing where the values come from rather than just what they are.

Quick Reference Card

Cut this page out and tape it to your monitor. Or don't — it will be here when you need it.

Arithmetic & Functions

Command / Syntax	What it does
+ - * /	Basic arithmetic
( )	Grouping / precedence
1.5e-6 or 200e9	Scientific notation
sin(x) cos(x) tan(x)	Trig (radians)
sqrt(x) abs(x)	Root, absolute value
log(x) exp(x)	Natural log, exponential
floor(x) ceil(x)	Round toward zero / away

Variables

Command / Syntax	What it does
$name = expr	Assign a value to a variable
$name	Retrieve a variable's value
vars	List all scalar variables
mats	List all matrix variables

Engineering Commands — Quick Syntax

Command / Syntax	What it does
reynolds V=<> D=<> nu=<>	Reynolds number → $Re
darcy f=<> L=<> D=<> V=<>	Pipe head loss → $darcy_hf
bernoulli ... solve=<var>	Bernoulli equation → $bern_P1/V1/z1/P2/V2/z2
orifice Cd=<> A=<> h=<>	Orifice flow → $orifice_Q
manning n=<> R=<> S=<> [A=<>] [D=<>]	Open channel flow → $manning_V
manning ?	Roughness coefficient table
beam type=ss|cant load=point|udl P=<>|w=<> L=<> E=<> I=<>	Beam deflection → $beam_delta
stress F=<> A=<> [E=<>] [L=<>]	Axial stress → $stress_sigma
mohrs sx=<> sy=<> txy=<>	Mohr's circle → $mohrs_s1/s2/tmax
idealgas P=<> V=<> n=<> T=<> solve=<var>	Ideal gas law → $gas_P/V/n/T
heatcond k=<> A=<> L=<> T1=<> T2=<>	Heat conduction → $heatcond_Q
specheat m=<> c=<> dT=<>	Specific heat → $heat_Q

Matrices

Command / Syntax	What it does
$A = [1 2; 3 4]	Create a 2×2 matrix
$A + $B	Element-wise addition
$A * $B	Matrix multiplication
transpose($A)	Transpose
inv($A)	Inverse (square only)
det($A)	Determinant
scale($A, 2.5)	Multiply all elements by 2.5

Signal Processing

Command / Syntax	What it does
load file.csv [col=N] [var=name]	Load CSV column into matrix variable
fft $signal [var=prefix]	FFT → $fft_re, $fft_im, $fft_mag, $fft_phase
ifft $fft_re $fft_im	Inverse FFT → $ifft_out

Shell

Command / Syntax	What it does
?	Full command reference
version	Parse version number
exit	Close Parse (Mathrix does this automatically)
# text	Comment — ignored by Parse

Thank you for reading the entire documentation.

You are in a very small and distinguished club.

You must also be very bored to get this far.

LICENSE INFORMATION

Mathrix and Parse mathematical equation and graphing tool.
Copyright (C) 2026 Bryan Wilcutt

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.